Understanding the DeepONet Robot-Arm Project

The question, the surrogate, the training, the data and the outcome

MMA3001 - Numerical Methods and Machine Learning - Monash University - 2026

How to read this document. It is written so that someone who has not seen the code can understand the entire project: what problem is being solved, what a 'surrogate' is, which numbers go in and come out, what exactly is being trained, how the training data was made, and what the finished project will show. Every figure marked *real* was produced by running the project's own code (`src/dynamics.py`, `src/solvers.py`, `src/data.py`) and its saved dataset. Diagrams are explanatory sketches.

Where the project actually is today (12 Oct 2026)
Built and working: the arm physics, the RK4 and RK45 solvers, the seeded dataset (2,200 simulated trajectories saved to `data/dataset.npz`), and an interactive visualiser.
Not written yet: the neural networks (`model.py`), training (`train.py`), evaluation and benchmarking - these files are empty placeholders. **So there are no accuracy or speed-up results yet.** Sections 4-5 describe the design the README commits to, with sizes marked as *proposals*, and Section 7 explains exactly how the results will be measured. No result numbers in this document are invented.

Contents

#	Section	Answers the question
1	The whole project in one page	What is this, in plain words?
2	The question: a 2-link robot arm	What system are we modelling, and what changes?
3	What is a surrogate?	Why replace a simulator with a neural network?
4	What is a DeepONet?	Why this particular network, and what is the baseline?
5	What am I training?	Inputs, outputs, weights, loss, the training loop
6	The data	How 2,200 trajectories were made, split and checked
7	The outcome	What gets measured, what success looks like, what to expect
8	Honest caveats and next steps	What could go wrong, what is left to do
9	Glossary	Every term in one place

1. The whole project in one page

The one-sentence version. A simulator can tell you how a robot arm moves, but it has to be re-run from scratch for every new payload, friction level and motor command. This project teaches a neural network (a **DeepONet**) to predict the arm's motion directly, then measures honestly how accurate and how fast that network is compared with the real simulator.

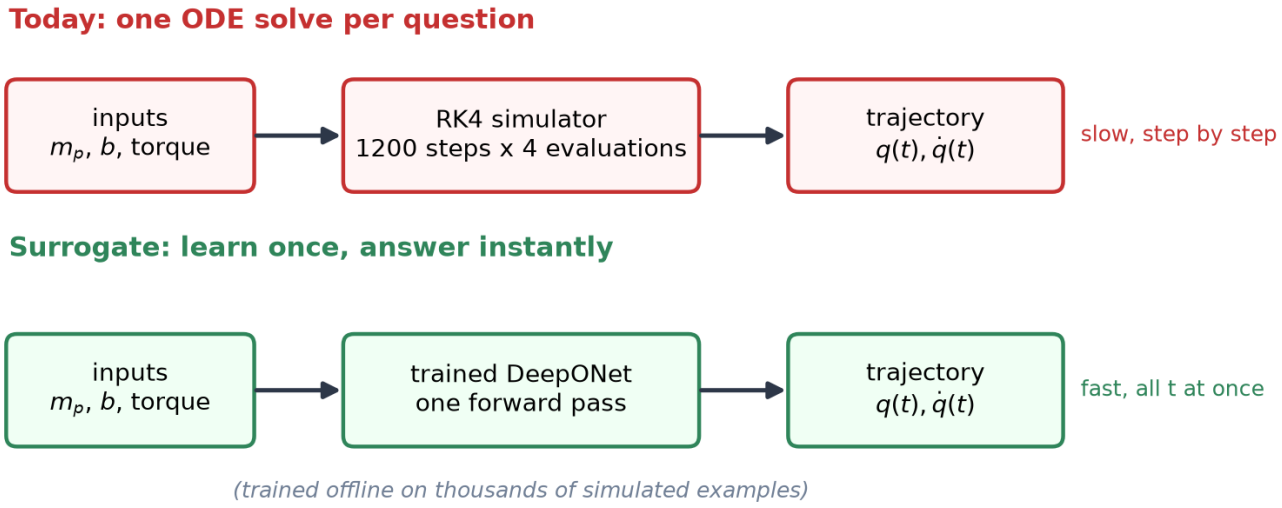


Figure 1. The idea. The simulator (top) computes the motion step by step. The surrogate (bottom) has seen thousands of simulated examples and answers in one go.

The story in five steps

- **Model the arm.** Write the physics of a 2-link planar robot arm (equations of motion with gravity, payload and friction). [done]
- **Solve it accurately.** Use a hand-written Runge-Kutta 4 (RK4) solver as the *ground truth*, and check it against theory (energy, convergence order). [done]
- **Make data.** Randomly pick payloads, frictions and motor torques; run RK4 on each; save 2,000 in-range + 200 out-of-range trajectories. [done]
- **Train.** Teach a DeepONet (and a plain MLP as a baseline) to map (payload, friction, torque) to the arm's trajectory. [to do]
- **Judge.** Test on unseen in-range and out-of-range cases; report error, error-versus-time, physical consistency and speed. [to do]

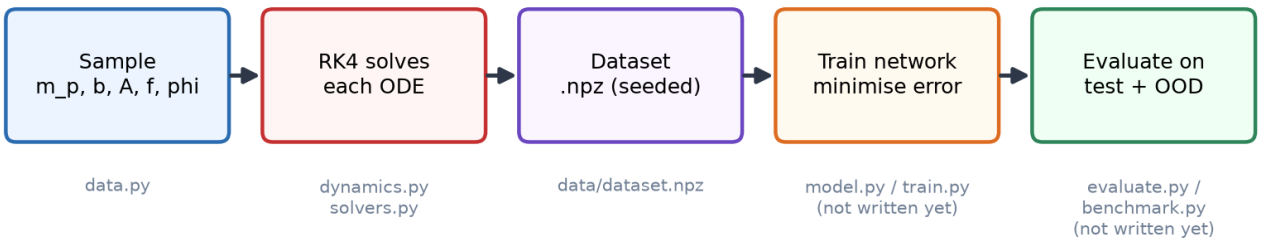


Figure 2. The project pipeline and the source file behind each stage.

Why is this a reasonable project?
The brief is not about complexity - it is about doing a small thing *rigorously*: a clear problem definition, a trustworthy ground truth, a fair baseline, honest validation (including cases outside the training range) and an honest discussion of limits. A 2-link arm is simple enough to verify by theory yet nonlinear enough (it can behave chaotically) to make the surrogate task non-trivial.

2. The question: a 2-link robot arm

2.1 The physical system

Imagine a robot arm with two rigid links and two motorised joints, mounted at a fixed base and moving in a **vertical plane** - so gravity is always trying to pull it down. The shoulder joint has angle q_1 (measured from the horizontal), the elbow joint has angle q_2 (measured relative to link 1). Each joint has a motor that applies a torque (a twisting force) tau_1, tau_2. A **payload** - a point mass m_p - sits at the tip of link 2.

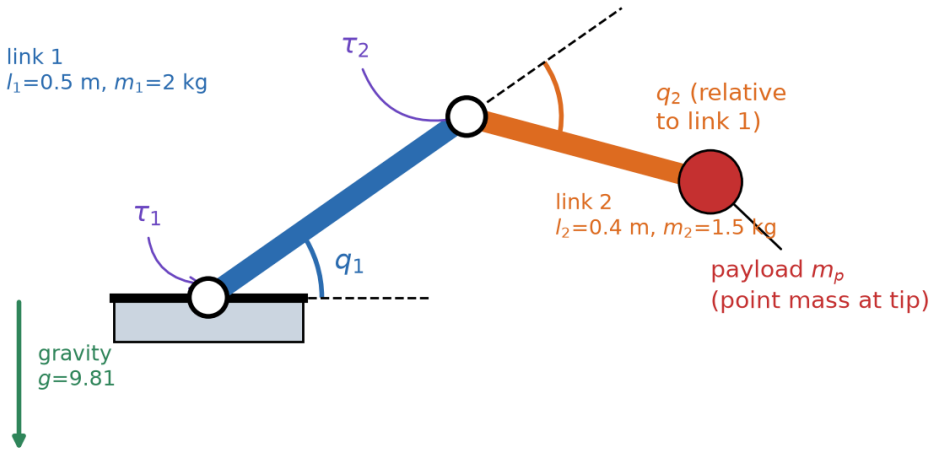


Figure 3. The arm. Link lengths and masses are fixed constants; the payload m_p, the friction b and the motor torques are what vary from case to case.

2.2 The equation that governs it

Newton's second law, written for rotating joints (the Euler-Lagrange form), says:

$$M(q) \cdot qddot + C(q, qdot) \cdot qdot + G(q) + b \cdot qdot = tau$$

Symbol	Plain-English meaning	Where in the code
M(q) qddot	**Inertia.** How hard it is to accelerate the arm. M depends on the pose q: an outstretched arm is harder to swing than a folded one.	`mass_matrix`
C(q, qdot) qdot	**Coriolis / centrifugal.** Velocity-dependent forces: swinging link 1 fast 'throws' link 2 around.	`coriolis_vector`
G(q)	**Gravity.** The torque needed just to hold the arm up; largest when the arm is horizontal.	`gravity_vector`

Symbol	Plain-English meaning	Where in the code
b qdot	**Viscous friction.** Resistance proportional to joint speed (b is the friction coefficient).	inside `state_derivative`
tau	**Motor torques** - the command we apply.	`torque_fn` in data.py

The code rearranges this to *qddot = M^{-1}(tau - C qdot - G - b qdot)* (solved with a linear solve, never an explicit inverse) and packages position and velocity together as the **state** *x = [q_1, q_2, $qdot_1$, $qdot_2$]*. Integrating *dx/dt = f(t, x)* forward in time gives the motion. The payload is folded into link 2's mass, centre of mass and inertia using the parallel-axis theorem (`combine_payload`).

2.3 What changes from case to case - the 'family' of arms

The project does not model *one* arm. It models a **family**, indexed by three things that can change:

Varies	Symbol	Range used for training	Out-of-range test
Payload at the tip	m_p (kg)	0 to 1	1 to 1.5 (heavier)
Joint friction	b (N m s/rad)	0.05 to 0.5	0.5 to 0.8 (stickier)
Motor torque over time	$tau_j(t) = A_j \sin(2 \pi f_j t + phi_j)$	A: 0-15 N m, f: 0.2-1.0 Hz, phase: 0-2 pi, independently for each joint	same as training

Fixed for every case: link lengths 0.5 m and 0.4 m, link masses 2 kg and 1.5 kg, gravity 9.81 m/s^2, a 3-second horizon, and the same starting condition (arm stretched out horizontally to the right, perfectly at rest: $x_0 = 0$).

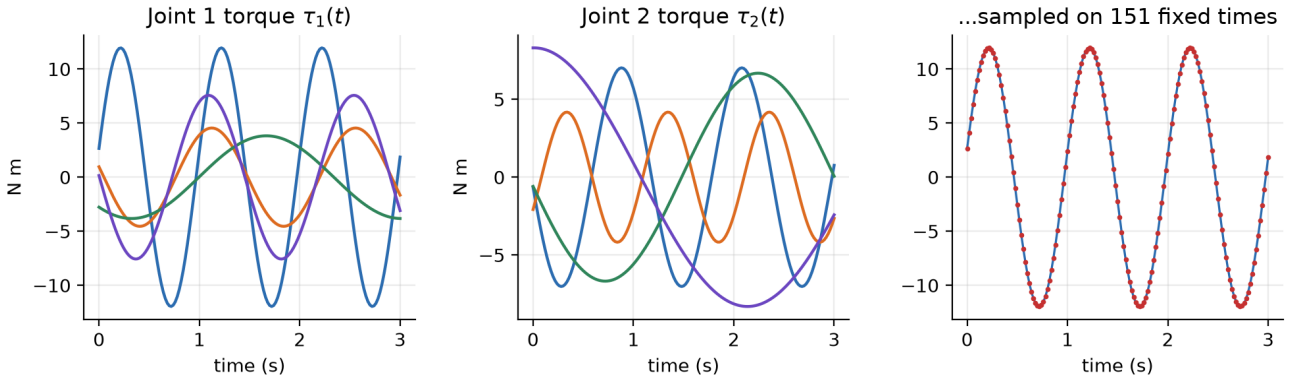


Figure 4 (real). The motor torque family: four random draws of the torque on each joint (left, middle), and how a torque curve is stored as 151 samples (right).

2.4 What the same arm does under different settings

The figure below applies the *same* motor commands to three versions of the arm. Payload and friction visibly change the entire motion - this is exactly the dependence the surrogate must learn.

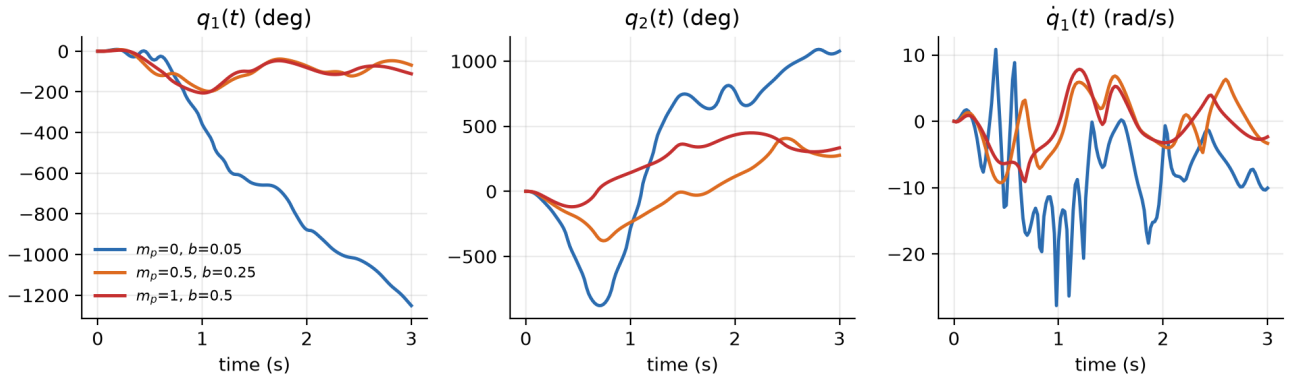


Figure 5 (real). Same torque commands, three (payload, friction) settings. The light, low-friction arm (blue) swings wildly; the heavy, high-friction arms (orange, red) are calmer.

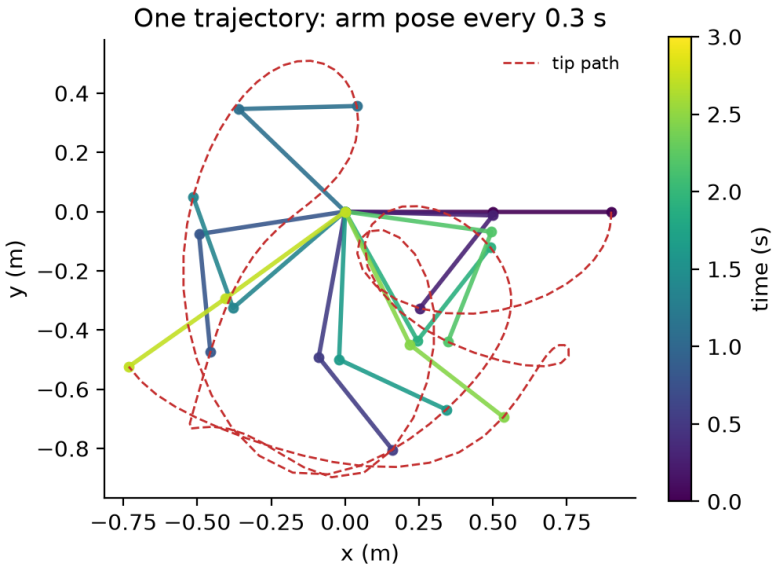


Figure 6 (real). One trajectory drawn as arm poses every 0.3 s (dark = early, bright = late), with the path of the tip.

The question, stated precisely
Given a payload m_p, a friction b and the motor torque curves $tau_1(t)$, $tau_2(t)$ over 0-3 s, **predict** the whole arm motion $q_1(t)$, $q_2(t)$, $qdot_1(t)$, $qdot_2(t)$ for every t in [0, 3] s - accurately, and much faster than integrating the ODE.

3. What is a surrogate?

3.1 The idea

A **surrogate model** (also called an emulator or a metamodel) is a cheap stand-in for an expensive computation. You run the expensive thing - here, the RK4 simulator - a few thousand times *once*, record inputs and outputs, and fit a fast model to those pairs. Afterwards you ask the fast model instead.

- **Analogy - a times-table vs. long multiplication.** Computing 7 x 8 by repeated addition always works but is slow. A student who has memorised the table answers instantly - but only for questions resembling what they memorised, and sometimes wrongly.
- **Analogy - a weather model.** A full atmospheric simulation takes hours; a learned emulator can give a rough forecast in seconds.
- **Here:** the 'expensive computation' is stepping the arm's ODE 1,200 times. The surrogate is a neural network that outputs the entire trajectory directly.

3.2 Why bother, for a robot arm?

The README names the use case: **fast parameter sweeps** - design studies (what payload can this arm handle?) and controller tuning (try thousands of torque profiles). These need the same simulation re-run an enormous number of times, so per-query cost matters.

> **A real measurement from this project**
> One ground-truth trajectory (1,200 RK4 steps, 4 derivative evaluations each, 4,800 evaluations in total) takes about **0.09 seconds** in this project's pure Python/NumPy implementation (5-run average on the development PC). A sweep of 100,000 settings is therefore about 2.5 hours. A neural network can process large batches of settings in parallel, which is the source of any speed-up - **but whether the speed-up is real, and how big, is exactly what** `benchmark.py` **must measure.** It is a hypothesis, not a result yet.

3.3 The trade-off every surrogate makes

	RK4 simulator (ground truth)	Neural-network surrogate
Accuracy	Controlled by step size; 4th-order accurate (verified, Section 6.4)	Approximate; error depends on training data and network - must be measured
Speed per query	~0.09 s (measured), sequential in time	Expected much faster, esp. in batches (to be benchmarked)
Works outside the training range?	Yes - physics is physics	Not guaranteed - can fail quietly (hence the out-of-range test set)
Respects physics?	Yes by construction (energy balance etc.)	Only if it happens to learn it - checked via energy drift
Up-front cost	None	Generate data + train (a one-off cost)

So the project is not 'neural networks are better'. It is: *how much accuracy do we give up, how much speed do we gain, and where does it break?*

4. What is a DeepONet?

4.1 Functions in, functions out

Ordinary machine learning maps a list of numbers to a list of numbers. Here the input includes a *function* (the torque curve over time) and the output is a *function* (the motion curve over time). A mapping from functions to functions is called an **operator**; learning one is **operator learning**. The **DeepONet** (Lu et al., 2021) is a network architecture designed for it, resting on the universal approximation theorem for operators.

In this project the operator is:

$$\mathbf{(m_p,\ b,\ tau_1(.),\ tau_2(.))} \quad \mathbf{\text{---->}} \quad \mathbf{x(.)} \quad \text{where } x(t) = [q_1,\ q_2,\ qdot_1,\ qdot_2](t)$$

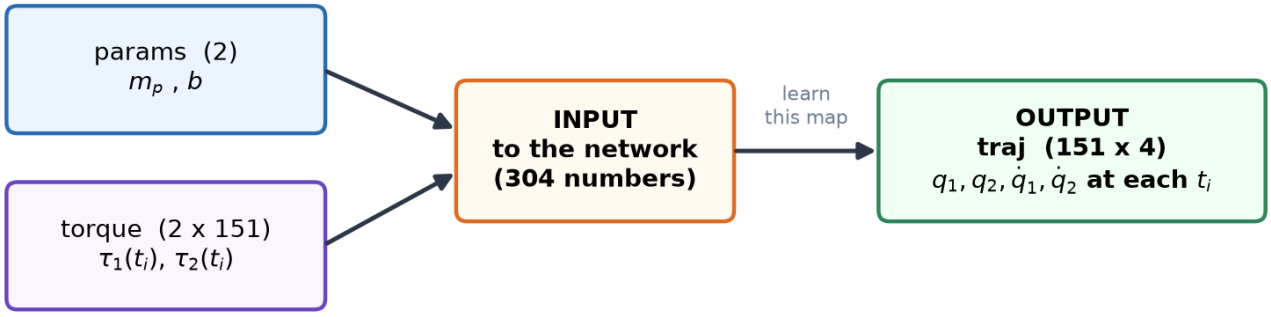


Figure 7. One sample in network terms: 304 numbers in (2 parameters + torque sampled at 151 times for 2 joints), a 151 x 4 table of states out.

4.2 The DeepONet design: Branch x Trunk

A DeepONet splits the job in two, with two small networks:

- **Branch network - 'what is the situation?'** It reads everything about the case (m_p, b and the sampled torque curves) and compresses it into a list of *p* numbers $b_1 ... b_p$, the *coefficients*.
- **Trunk network - 'at what time?'** It reads only the query time *t* and produces *p* numbers $t_1 ... t_p$, learned *basis functions* of time.
- **Combine:** the prediction at time t is the dot product $sum_k\ b_k\ t_k$ (one such combination per output channel).

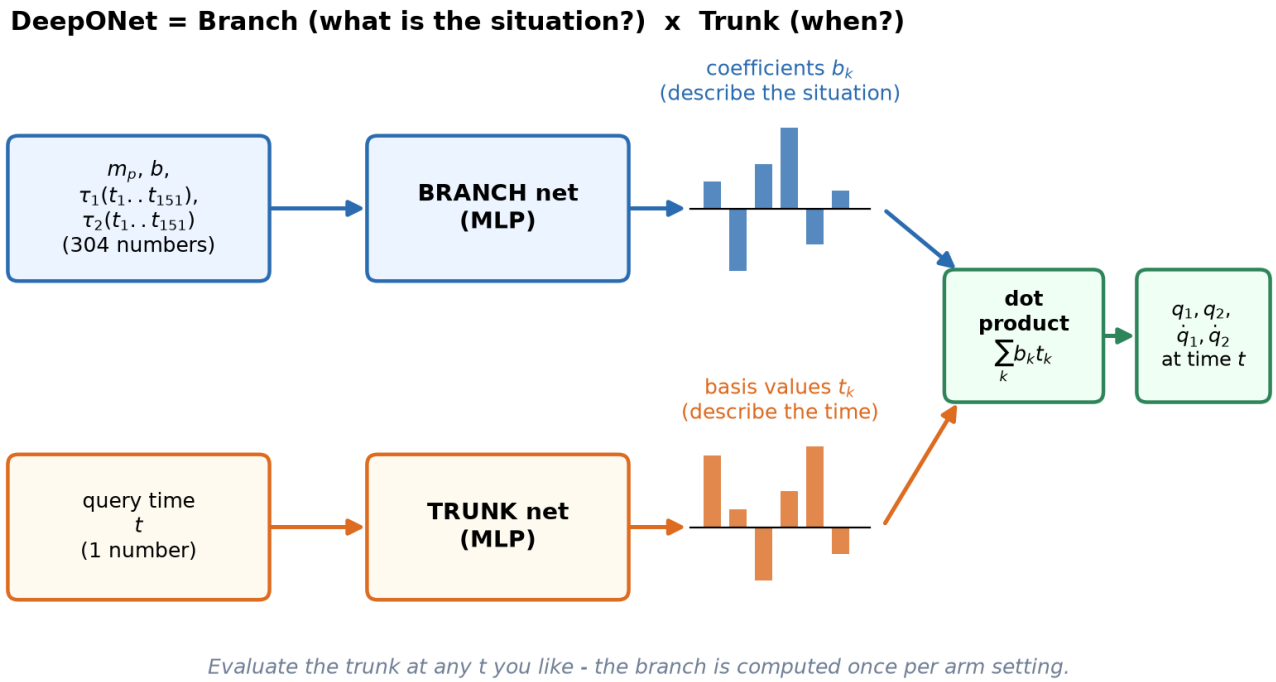


Figure 8. DeepONet. The branch encodes the situation once; the trunk can then be evaluated at any time t. The bar charts are illustrative values.

> **Intuition: a recipe and a set of ingredients**
> Think of any trajectory as a mixture of building-block curves over time (the trunk's t_k). The branch decides *how much of each block* to use for a particular arm setting (b_k). Different payloads and torques just give different mixing amounts. This is similar in spirit to a Fourier series, except the building blocks are *learned* from data rather than fixed sines.

A practical bonus: because time enters only through the trunk, the trained model can be asked about any time between 0 and 3 s, not only the 151 grid points it was trained on.

4.3 The baseline: a plain MLP

A result is only meaningful against a fair comparison. The baseline is a **multilayer perceptron (MLP)** - a standard fully connected network - given exactly the same information, but with the time t simply appended to the other inputs and one network producing the answer.

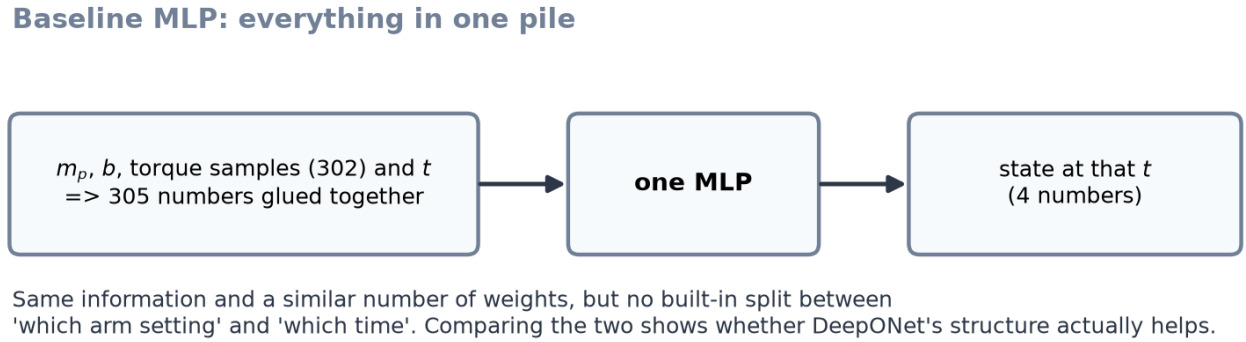


Figure 9. The MLP baseline treats the time as just another input number.

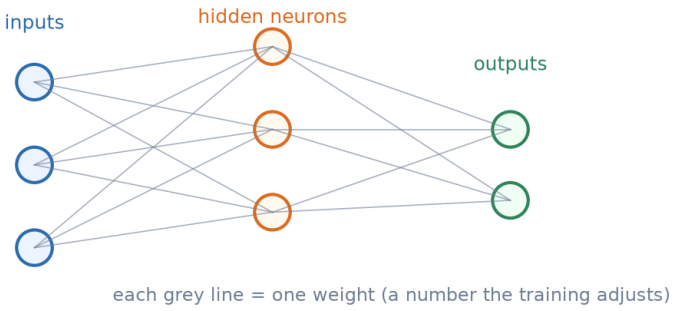


Figure 10. What a network is: layers of simple units joined by weighted connections. Training adjusts the weights.

If the DeepONet beats an MLP of similar size, that supports the claim that its branch/trunk *structure* helps. If it does not, that is also a legitimate finding to report and explain.

5. What am I training?

> **Status note**
> `model.py` and `train.py` are empty. The input/output definition below comes from the README and the dataset. Network sizes, loss and optimiser are **proposals** (typical choices), not decisions already made - lock them when you write the code.

5.1 Exactly what goes in and what comes out

	DeepONet	MLP baseline
Input A	**Branch input (304 numbers):** m_p, b, 151 samples of tau_1, 151 samples of tau_2	**One input (305 numbers):** the same 304 numbers plus the query time t
Input B	**Trunk input (1 number):** query time t	-
Output	4 numbers at time t: q_1, q_2, $qdot_1$, $qdot_2$ (or all 151 times at once by evaluating the trunk on the grid)	same 4 numbers at time t
Ground truth	the RK4 trajectory stored in the dataset (151 x 4)	same

One training sample therefore yields 151 (input, target) pairs - one for each time on the grid - so 1,400 training trajectories give 211,400 labelled points to learn from.

5.2 What are the 'weights'?

Each network is a long chain of arithmetic: multiply inputs by numbers, add, pass through a non-linear function, repeat. The multipliers and offsets are the **weights** (parameters). At the start they are random, so predictions are nonsense. **Training is the process of adjusting the weights until predictions match the RK4 trajectories.**

Illustrative size (proposal). A branch net of 304-128-128-128-256 and a trunk net of 1-128-128-128-64 (p = 64 basis functions, 4 output channels) has roughly 146,000 weights. The MLP would be sized to have a similar count so the comparison is fair. The parameter count goes in the results table.

5.3 The loss: how 'wrong' is a prediction?

A **loss** is a single number measuring the total error. The natural choice is the mean-squared error between predicted and RK4 states, computed on **standardised** channels (subtract the mean, divide by the standard deviation) so that the huge velocity values do not drown out the angles. The headline *evaluation* metric, however, is the **relative L2 error** of the README:

relative L2 error = || predicted trajectory - true trajectory || / || true trajectory ||

It reads as 'the error as a fraction of the size of the signal': 0.02 would mean 2% error.

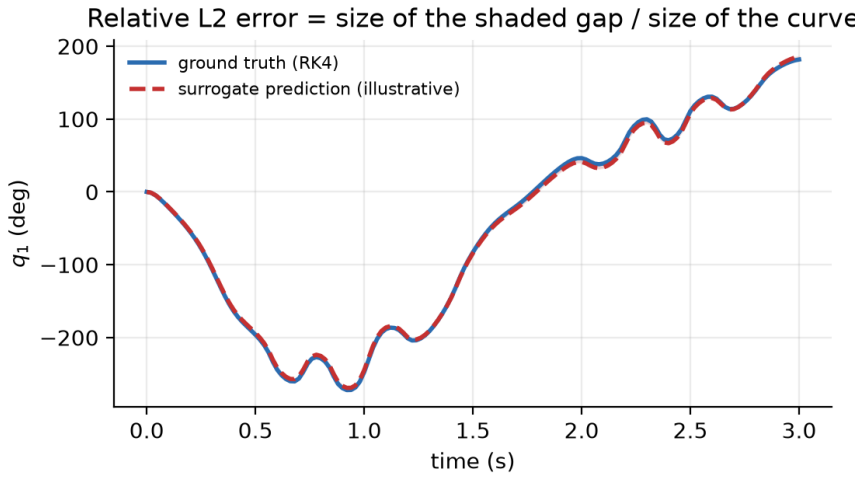


Figure 11 (illustrative prediction). The shaded gap between the surrogate and the truth, compared with the size of the curve itself, is the relative L2 error.

5.4 The training loop

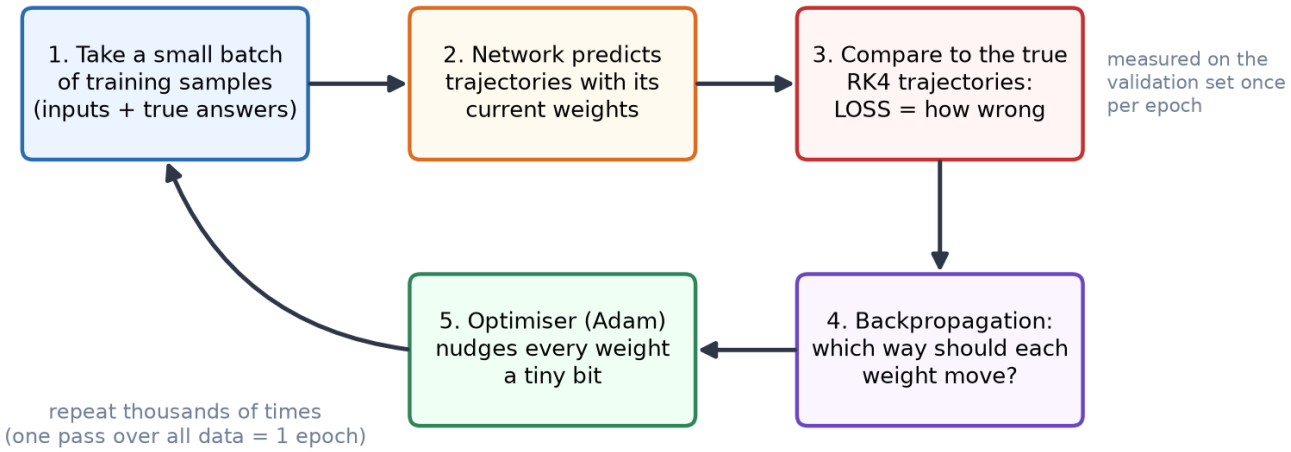


Figure 12. One round of training, repeated thousands of times. Adam is a popular optimiser that scales each weight's step sensibly.

- **Training set (1,400)** - the only data the weights ever learn from.
- **Validation set (300)** - watched during training to decide when to stop and to choose settings such as layer width and learning rate. This is how *over-fitting* (memorising the training data) is detected.
- **Test set (300)** - touched only once at the end, for the honest final score. The README states it is *never* used for model selection.
- **Out-of-range set (200)** - a stress test of extrapolation, also never trained on.

Hardware: CPU only. The README states that no GPU is needed; at this data size that is plausible.

6. The data

6.1 How one sample is manufactured

Every training example comes from the physics, not from a real robot. For each sample the code in `src/data.py` does the following:

- **Draw random inputs** from uniform distributions: m_p, b, and for each joint an amplitude A, frequency f and phase phi.
- **Build the torque curve** $\tau_j(t) = A_j \sin(2\pi f_j t + \phi_j)$ and sample it at 151 evenly spaced times between 0 and 3 s (every 0.02 s).
- **Solve the ODE with RK4** from the starting state $x_0 = 0$, using 8 internal sub-steps per 0.02 s (h = 0.0025 s, 1,200 steps), keeping every 8th point.
- **Store** params (2), torque (2 x 151) and trajectory (151 x 4).

The whole thing is **seeded** (seed 3001), so anyone can regenerate the identical dataset with `python -m src.data`.

6.2 The dataset at a glance (real)

Split	Samples	Purpose	Parameter ranges
train	1,400	Fit the weights	m_p in [0,1], b in [0.05,0.5]
val	300	Choose settings, early stopping	same
test	300	Final in-range score	same
out-of-range (OOD)	200	Extrapolation stress test	m_p in [1,1.5], b in [0.5,0.8]

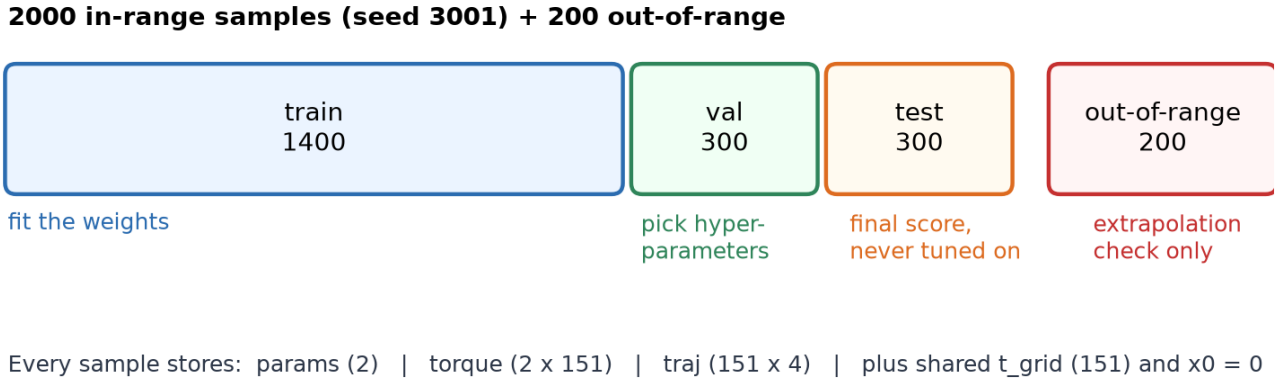


Figure 13 (real). The dataset split. All sets are saved in one file, data/dataset.npz.

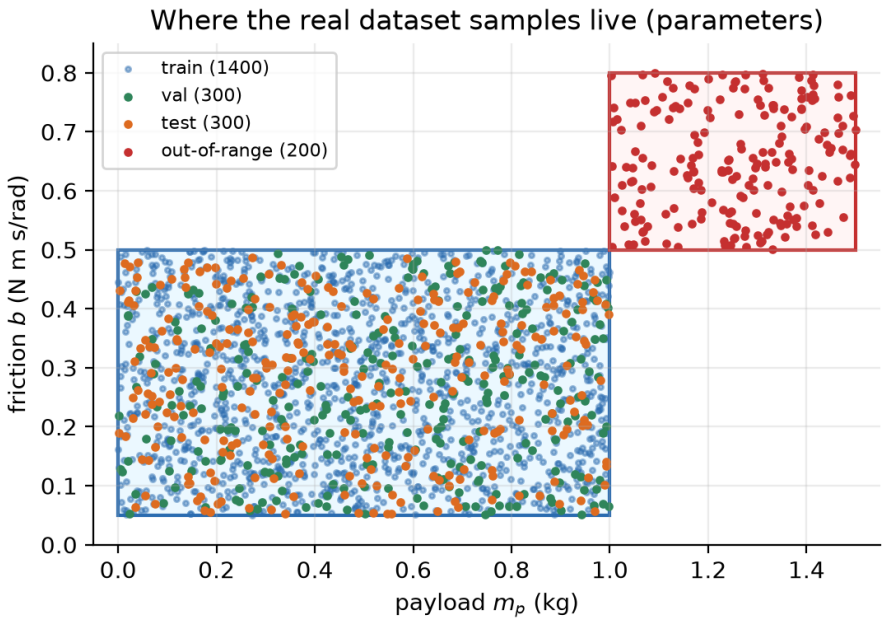


Figure 14 (real). Every dot is one sample, plotted by its payload and friction. The red OOD cloud lies entirely outside the blue training region - that is what 'out-of-range' means.

Why a 70 / 15 / 15 split? The network should be scored on data it has never seen, otherwise a model that memorised the answers would look perfect. Separating validation (for tuning) from test (for the final grade) stops the tuning from quietly leaking into the final score.

6.3 What the trajectories look like

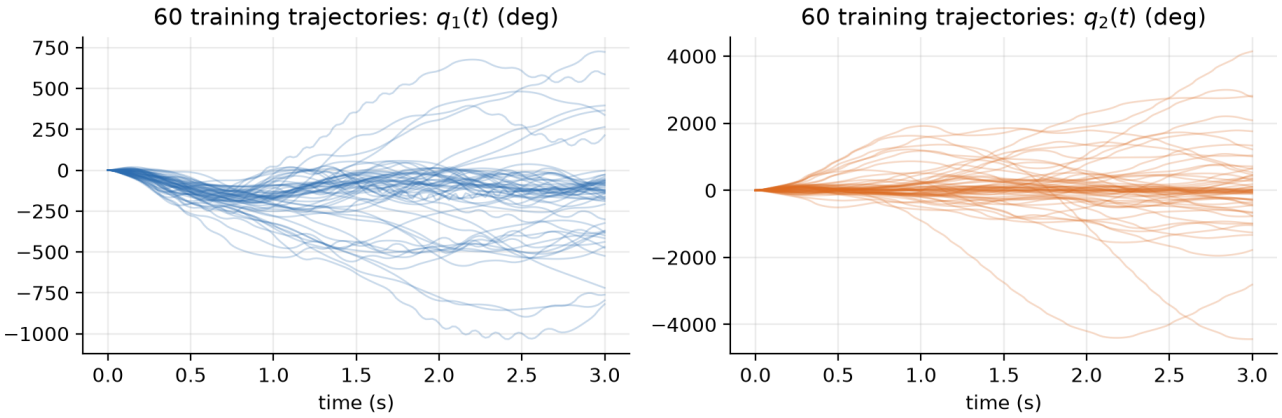


Figure 15 (real). Sixty random training trajectories. Starting from the same pose at rest, they fan out widely - the network must capture all of this variety.

Important finding from the real dataset
The motion is much wilder than 'a robot reaching for something'. Torques up to 15 N m on a light forearm cause the joints to **spin through multiple full revolutions**. In the training set, **66%** of trajectories have the elbow turn through more than one full rotation (median peak about 480 degrees; the extreme is over 7,500 degrees, i.e. 21 turns) and the peak elbow speed reaches **196 rad/s** (median 33 rad/s). The shoulder exceeds a full turn in 32% of samples. This is a double-pendulum-like, partly chaotic regime, where tiny input changes can produce very different outcomes. It makes the learning problem **harder** than the README's 'robot arm' picture suggests, and it is worth discussing in the report (Section 8).

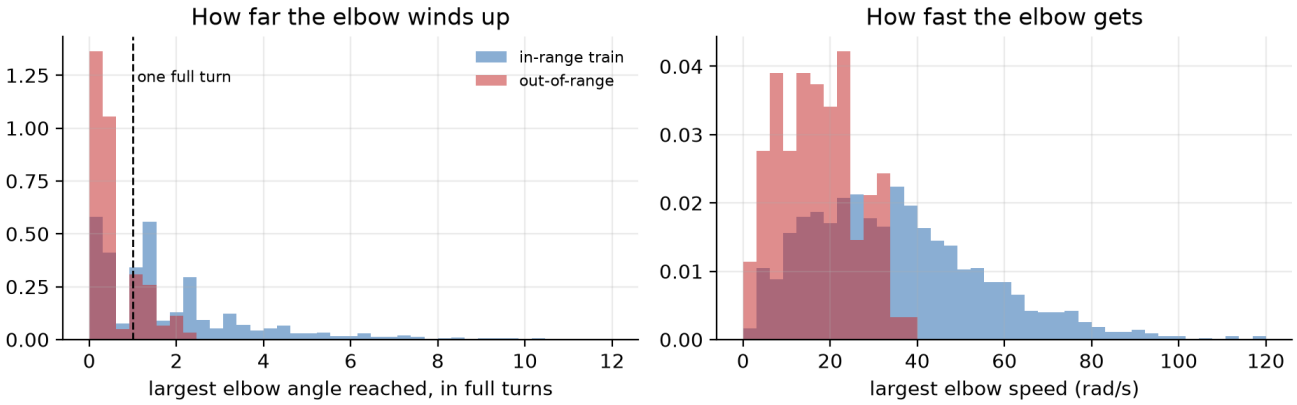


Figure 16 (real). How far (left) and how fast (right) the elbow goes. Note that the out-of-range set (red) is actually calmer than the training set.

The right-hand figure also reveals something subtle: the out-of-range cases are **not** more violent than the training ones - heavier payload and extra friction damp the motion (only 23% of OOD samples exceed a full elbow turn, against 66% in training). So the OOD test probes extrapolation in *parameters*, but not necessarily in *motion magnitude*. A surrogate could do well on OOD for that reason; interpret it carefully.

6.4 Can the ground truth be trusted?

A surrogate can only be as good as the data it imitates, so the solver itself must be validated first. Two checks were re-run for this document:

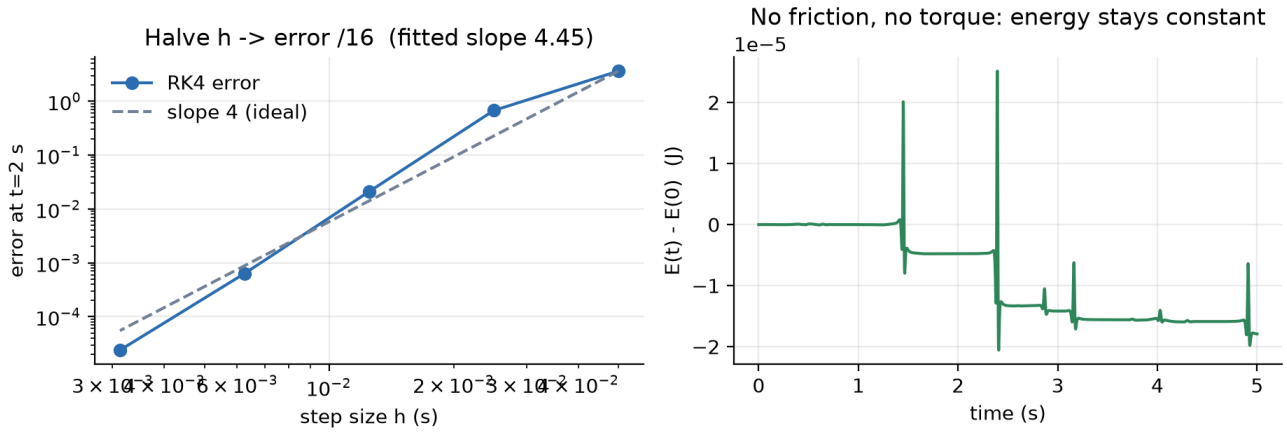


Figure 17 (real). Left: halving the RK4 step cuts the error by about 16x (a fitted slope of about 4.4 on the log-log plot, close to the theoretical 4). Right: with friction and torque switched off, total mechanical energy stays constant - it changes by only 0.00005 J out of 3.86 J over 5 s.

- **Convergence order:** a 4th-order method should lose 4 digits of error per factor of 10 in step size. The measured slope was about 4.4 (slightly above 4 because the error is still settling into its asymptotic regime at coarse steps).
- **Energy conservation:** a frictionless, unforced arm obeys energy conservation; the equations in the code reproduce it to about 1e-5 relative accuracy.
- **Planned further checks in the README:** the energy balance with friction and torque (dE/dt = tau . qdot - b |qdot|^2), the single-pendulum limit, and agreement with SciPy's `solve_ivp`. An adaptive RK45 solver already exists and can be overlaid in the visualiser.
- **A stability warning I saw while checking:** with a coarse step of 0.2 s, RK4 on this arm *blows up* (the result becomes NaN). The dataset's 0.0025 s step is far finer and its 2,200 trajectories contain no NaN values, but it shows why the step size must be justified.

> **About the unit tests**
> The repository contains `tests/test_dynamics.py` and `tests/test_smoke.py`. pytest was not installed in the Python environment used to write this document, so I did **not** run them here and make no claim about whether they pass. Run `pytest` in your project environment before quoting test results.

7. The outcome

7.1 What the finished project will deliver

- A **verified ground-truth simulator** (RK4, cross-checked with RK45 and SciPy) with unit tests.
- A **reproducible, seeded dataset** of 2,200 trajectories.
- A trained **DeepONet** and a trained **MLP baseline**.
- An **evaluation** showing accuracy in-range and out-of-range, how error grows over time, and whether predictions obey physics.
- A **benchmark** of runtime, memory and arithmetic cost (FLOPs) for RK4, `solve_ivp`, the MLP and the DeepONet.
- A report with a candid **limitations** discussion and an AI-use reflection (see AI_LOG.md).

7.2 How success will be measured

Question	Measurement	How to read it
Is it accurate on cases like the training data?	Relative L2 error on the 300 test trajectories (mean, and spread/worst-case)	Lower is better; compare DeepONet vs MLP
Does it generalise beyond the training range?	Relative L2 error on the 200 OOD trajectories	Expect worse than in-range; how *much* worse is the finding
Where in time does it go wrong?	Error as a function of t (0 to 3 s)	Errors typically grow over time as small mistakes compound
Is the output physically sensible?	Energy drift of the predicted trajectories (energy() from dynamics.py applied to the predictions)	A network can have low error yet violate energy balance
Is it actually faster?	Seconds per trajectory, memory, FLOPs; including a batched comparison	Speed-up must be weighed against the accuracy loss
Is the comparison fair?	Similar parameter counts, same data, same split, same seed	Lets you attribute differences to the architecture

7.3 The results table (to be filled in)

This is the README's planned table. Every 'TBD' is a number that does not exist yet:

Method	Rel. L2 (in-range)	Rel. L2 (out-of-range)	Params	Train time	Time / trajectory
RK4 (ground truth)	n/a	n/a	n/a	n/a	~0.09 s (measured in Python)
solve_ivp	TBD	TBD	n/a	n/a	TBD
MLP baseline	TBD	TBD	TBD	TBD	TBD
DeepONet	TBD	TBD	TBD	TBD	TBD

7.4 What I would expect (hypotheses, not results)

These are reasoned guesses to test, written down *before* the experiment so they can be honestly confirmed or refuted:

- **Accuracy will degrade with time.** Chaotic, spinning motion amplifies small errors; late-time predictions should be worse than early ones.
- **The heavy, low-friction, high-torque cases will be hardest** - they spin the most (Figure 5, blue).
- **OOD error will exceed in-range error** - but, from Figure 16, perhaps not dramatically, because OOD motion is gentler.
- **A DeepONet may or may not beat the MLP.** At this small scale the structured design could help, or the two could tie; either result is reportable.
- **Energy drift may be non-zero** even for accurate networks, since nothing in the loss enforces physics.

A key honest framing for the final report: *validation shows that the network matches the model, not a physical robot.* All data come from the project's own equations.

8. Honest caveats and next steps

8.1 Limits of the study (from the README, plus findings here)

- **Simulation only.** No physical robot; the surrogate imitates the equations, not reality.
- **Narrow scope:** planar 2-link arm, vertical plane, viscous friction only (no Coulomb friction), fixed 3 s horizon, and always the **same starting pose** ($x_0 = 0$).
- **The torque family is smooth and low-dimensional.** Each torque curve is really described by just 6 numbers (A, f, phi per joint), so the true input space has only 8 degrees of freedom, even though the branch sees 304 numbers. Torques of a different shape (steps, ramps, noise) are untested and should not be claimed.
- **Spinning, partly chaotic motion** (Section 6.3) makes long-horizon accuracy genuinely hard; consider whether to shorten T, lower the amplitude cap, or accept and discuss it.
- **Angles are unbounded.** The elbow reaches thousands of degrees, so the network predicts a raw winding angle rather than a wrapped one - a deliberate and worthwhile thing to document.
- **The OOD set tests parameters, not violence of motion** (Section 6.3).

8.2 Decisions still open

Open item	Why it matters
Horizon T (README says 2 to 4 s, code uses 3 s) and parameter ranges are marked provisional	Changing them changes the dataset; freeze before training and record in the README
Torque amplitude cap (15 N m)	Chosen so the motor can compete with gravity (about 12.3 N m at the start pose), but it also drives the spinning
Network widths, depth, p, learning rate	Choose with the *validation* set only
Input validation (NaN, negative mass, out-of-range warnings)	Listed in README as to be implemented
Random seed recorded in README ('TBD')	Code uses 3001; update the README so results are reproducible

8.3 Suggested order of work

- Write `model.py`: DeepONet and MLP in PyTorch (install PyTorch first; it is in requirements.txt but absent from the Python used to prepare this document).
- Write `train.py`: standardise data, mini-batch Adam, validation-based early stopping, log the loss curves.

- Train both models, tune on validation, then freeze.
- Write `evaluate.py`: relative L2 (in-range, OOD), error versus time, energy drift, best/worst examples; plug the models into `viz.SURROGATES` to see predicted arms next to RK4.
- Write `benchmark.py`, fill in the results table, finish the limitations section and the AI-use reflection.

9. Glossary

Term	Meaning
ODE	Ordinary differential equation: a rule giving the rate of change of the state; solving it means predicting the future from the present.
RK4 / RK45	Runge-Kutta methods for solving ODEs step by step. RK4 uses a fixed step and is 4th-order accurate; RK45 (Dormand-Prince) adapts its step size using an error estimate.
State	The minimum information needed to continue the motion: [q1, q2, qdot1, qdot2].
Torque	A twisting force applied by a joint motor (N m).
Payload	An extra mass carried at the tip of the arm.
Viscous friction	A resisting torque proportional to joint speed.
Surrogate / emulator	A fast learned model standing in for a slow simulation.
Operator	A mapping from functions to functions (here: torque curves to motion curves).
DeepONet	A network with a branch (encodes the input function) and a trunk (encodes the query location/time) combined by a dot product.
MLP	Multilayer perceptron: a standard fully connected neural network.
Weights	The adjustable numbers inside a network; training sets them.
Loss	A number measuring how wrong the predictions are; training minimises it.
Epoch	One full pass through the training data.
Over-fitting	Memorising the training data and failing on new data; detected with the validation set.
Validation / test set	Held-out data for tuning (validation) and for the final unbiased score (test).
Out-of-range (OOD)	Test cases from outside the parameter ranges seen in training.
Relative L2 error	Size of the prediction error divided by the size of the true signal.
Energy drift	How much a trajectory's total mechanical energy changes in ways physics forbids.
Seed	A starting number that makes 'random' generation exactly repeatable.
FLOPs	Floating-point operations: a hardware-independent count of arithmetic work.

Reference. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3, 218-229.